

华中科技大学

# 课程实验报告

课程名称： 深度学习

实验名称： 实验一

院 系： 计算机科学与技术学院

专业班级： 本硕博 2301 班

学 号： U202315763

姓 名： 王家乐

2025 年 12 月 16 日

# 1. 任务要求

实验旨在通过设计并训练一个至少包含一层隐藏层的多层感知机（Multilayer Perceptron, MLP），完成对 MNIST 手写数字数据集的分类任务。并且通过实验理解多层感知机的网络结构及其在图像分类任务中的应用原理，重点掌握神经网络中前向传播与反向传播的计算流程，包括各层输入输出、损失函数的计算、梯度的反向传播以及模型参数的更新机制。同时，通过调整网络层数、神经元数量及激活函数，分析不同网络结构和参数设置对模型性能的影响。

## 2. 实验设计

### 2.1 实验环境

| 项    | 参数                                   |
|------|--------------------------------------|
| 操作系统 | Windows11                            |
| CPU  | 12th Gen Intel(R) Core(TM) i7-12700H |
| RAM  | 16GB                                 |
| 编程语言 | Python3.10.18                        |
| 平台   | VScode & Jupyter Notebook            |

### 2.2 相关依赖

| 依赖项        | 版本     |
|------------|--------|
| torch      | 2.8.0  |
| pandas     | 2.3.1  |
| numpy      | 2.2.6  |
| matplotlib | 3.10.5 |
| tqdm       | 4.67.1 |

### 2.3 数据集加载和处理

本实验采用 MNIST 手写数字数据集作为训练与测试数据。MNIST 数据集包含 60000 张训练样本和 10000 张测试样本，每张样本大小为  $28 \times 28$  的灰

度图像，对应的类别标签为 0 至 9，共 10 个数字类别。数据在加载后分别以 train\_data、train\_label、test\_data 和 test\_label 的形式存储，其中数据和标签均以 numpy.array 的形式进行处理，便于后续的数值计算与模型训练。

```
def load_mnist_numpy(root="data"):
    """加载 MNIST 数据集并转换为 Numpy 格式"""
    tfm = transforms.ToTensor() # 定义转换: [0,1], shape: [1,28,28]
    # 下载并加载训练集和测试集
    train_ds = MNIST(root=root, train=True, download=True, transform=tfm)
    test_ds = MNIST(root=root, train=False, download=True, transform=tfm)
    # 提取数据和标签并转换为 numpy 数组
    train_data = train_ds.data.numpy()
    train_label = train_ds.targets.numpy().astype(np.int64)
    test_data = test_ds.data.numpy()
    test_label = test_ds.targets.numpy().astype(np.int64)
    return train_data, train_label, test_data, test_label
# 加载数据
train_data, train_label, test_data, test_label = load_mnist_numpy(root="data")
```

采用 PyTorch 的 Dataset 接口自定义了 MNISTDataset 类，用于统一管理数据的加载与预处理过程。该类在初始化阶段接收原始图像数据和对应标签，并将标签类型转换为 int64，以满足后续分类任务与损失函数的要求。将原始的  $28 \times 28$  灰度图像转换为 float32 类型，并对像素值进行归一化处理，将其缩放到 [0, 1] 区间，以减小不同特征量纲带来的影响。随后将二维图像展平成长度为 784 的一维向量，以适配多层感知机模型对向量形式输入的要求。处理完成后，数据和标签均被转换为 PyTorch 的 Tensor 类型，便于后续通过 DataLoader 进行批量加载和模型训练。

```
class MNISTDataset(Dataset):
    """自定义 Dataset 类，用于 PyTorch 数据加载"""
    def __init__(self, data, labels):
        self.data = data
        self.labels = labels.astype(np.int64)
    def __len__(self):
        return len(self.data)
    def __getitem__(self, idx):
        # 数据预处理：归一化和展平
```

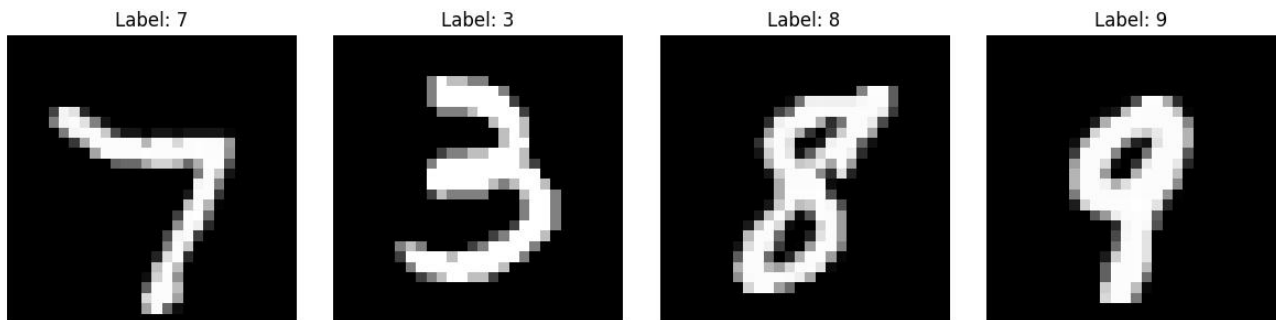
```

x = self.data[idx].astype(np.float32) / 255.0    # 归一化 (28,28) -> [0,1]
x = x.reshape(-1)                                # 展平 (784,)
y = self.labels[idx]
return torch.from_numpy(x), torch.tensor(y, dtype=torch.long)

# 实例化Dataset 对象
train_dataset = MNISTDataset(train_data, train_label)
test_dataset = MNISTDataset(test_data, test_label)

```

从数据集中随机挑选四张进行可视化如下图所示：



## 2.4 多层感知机设计

多层感知机是由若干全连接层组成的神经网络，每一层的输入维数必须与上一层的输出维数相等，层与层之间需要合适的激活函数进行非线性变换。本实验输入层的维数应为 784 ( $28 \times 28$ )，输出层的维数应为 10。默认的网络只有一个大小为 128 的隐藏层，使用 ReLU 激活函数。

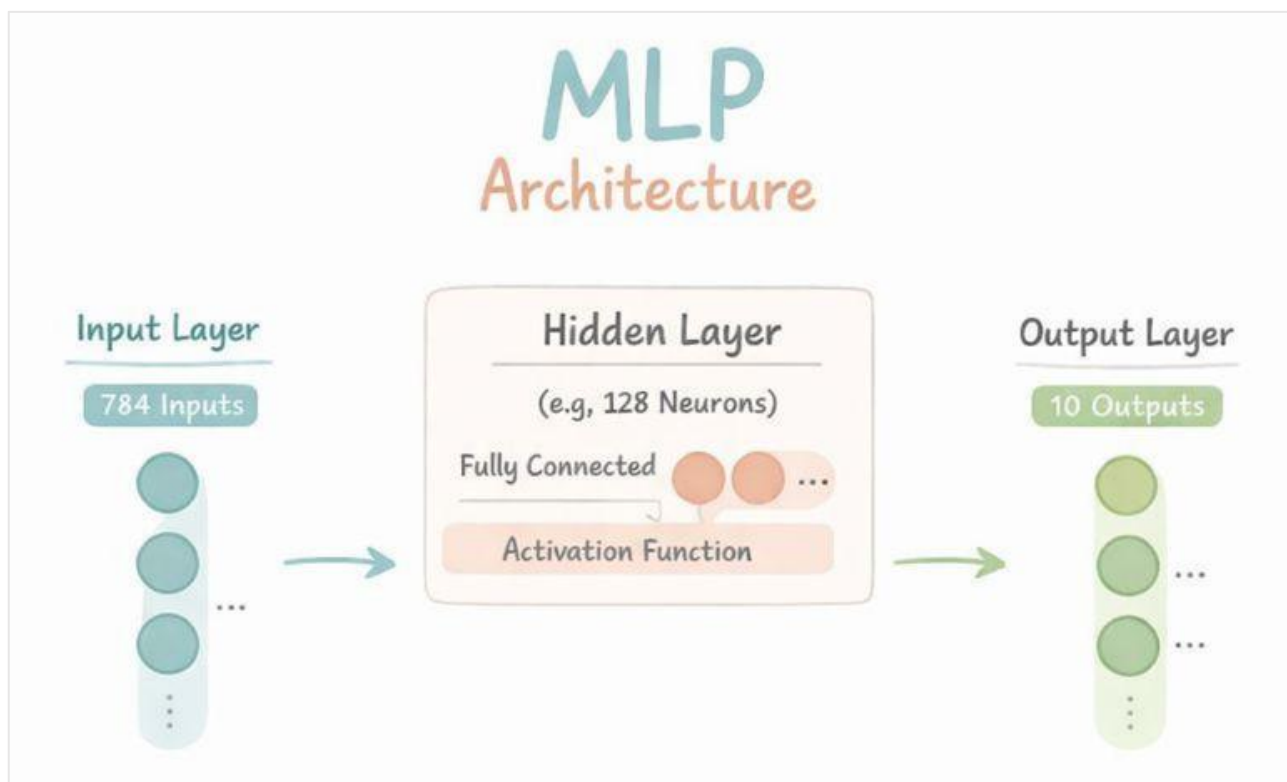
```

class MLP(nn.Module):
    """ 多层感知机 """
    def __init__(
        self,
        input_size=784,
        hidden_sizes=[128],
        output_size=10,
        activation='relu'
    ):
        super(MLP, self).__init__()
        # 构建网络层列表
        layers = []
        prev_size = input_size
        # 循环构建隐藏层
        for hidden_size in hidden_sizes:
            layers.append(nn.Linear(prev_size, hidden_size)) # 全连接层

```

```
# 根据配置选择激活函数
if activation == 'relu':
    layers.append(nn.ReLU())
elif activation == 'sigmoid':
    layers.append(nn.Sigmoid())
elif activation == 'tanh':
    layers.append(nn.Tanh())
elif activation == 'leaky_relu':
    layers.append(nn.LeakyReLU())
else:
    layers.append(nn.ReLU()) # 默认使用 ReLU
prev_size = hidden_size
# 添加输出层
layers.append(nn.Linear(prev_size, output_size))
# 将列表解包并封装为 Sequential 容器
self.network = nn.Sequential(*layers)
# 保存网络参数
self.hidden_sizes = hidden_sizes
self.activation = activation
```

本实验默认的网络结构如下图所示：



## 2.5 网络训练

首先根据配置信息（hidden\_sizes、activation）创建模型。

```
def create_model(cfg):  
    """ 根据配置字典创建模型实例 """  
    return MLP(  
        input_size=cfg.get("input_size", 784),  
        hidden_sizes=cfg.get("hidden_sizes", [128]),  
        output_size=cfg.get("output_size", 10),  
        activation=cfg.get("activation", "relu"),  
    )
```

接下来训练模型，最大迭代次数（num\_epochs）默认为 10，模型名称（name）、批大小（batch\_size）、和学习率（learning\_rate）从配置信息 config 中获取。使用 DataLoader 创建数据对象，本实验选用交叉熵（Cross Entropy）作为损失函数，选用 Adam（Adaptive Moment Estimation）作为优化器。训练过程中记录每个 epoch 在训练集和测试集上的损失和准确率，并将其写入 output 文件夹下的同名 log 文件中。此外，为了更为直观地反映网络训练全过程的性能变化和分类效果，在训练结束后通过 matplotlib 库对训练集和测试集的损失、准确率进行可视化展示。具体代码过长，这里不再展示。

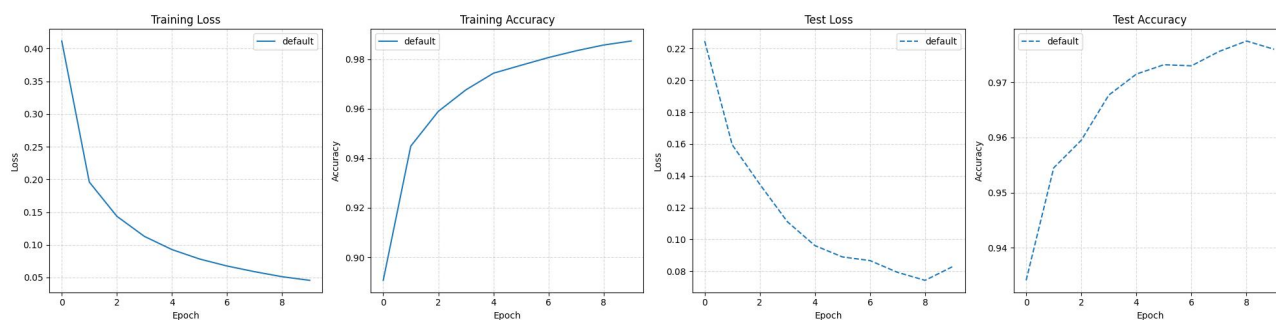
## 3. 训练结果与分析

### 3.1 默认网络结构

如 2.4 节所述，默认网络结构为 784（input\_size）\*128（hidden\_size）\*10（output\_size），采用 ReLu 激活函数，训练条件如下表所示：

| 参数            | 值     |
|---------------|-------|
| num_epochs    | 10    |
| batch_size    | 128   |
| learning_rate | 0.001 |

训练过程使用 tqdm 库通过进度条在控制台实时显示各项指标，以 epoch 为单位更新。训练结束后，得到训练过程的模型性能变化曲线如下图：

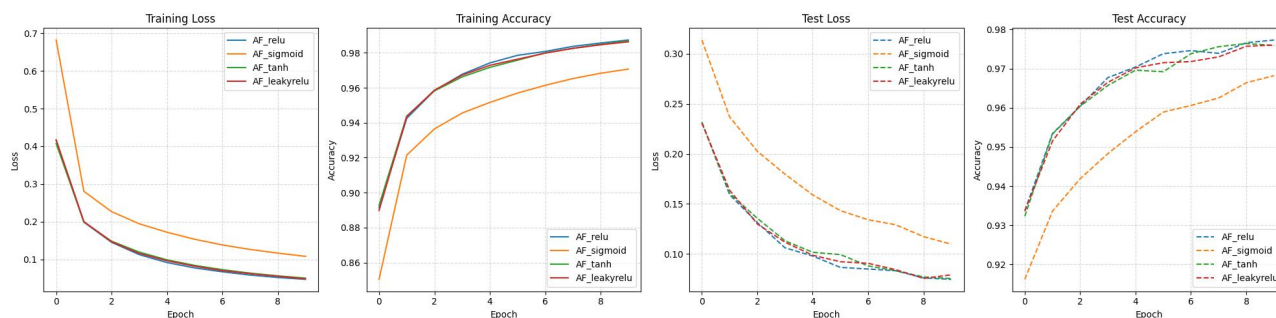


从实验结果可以看出，在默认的网络结构和训练条件下，模型的训练过程达到稳定且最终能收敛到较优的结果上（训练集准确率 98.73%，测试集准确率 97.60%）。接下来依次改变网络结构和训练条件。default.log 文件如下：

```
Epoch,Train Loss,Train Accuracy,Test Loss,Test Accuracy
1,0.4117,0.8907,0.2248,0.9341
2,0.1960,0.9449,0.1595,0.9545
3,0.1436,0.9589,0.1347,0.9595
4,0.1129,0.9676,0.1111,0.9677
5,0.0927,0.9743,0.0961,0.9715
6,0.0783,0.9775,0.0889,0.9732
7,0.0676,0.9807,0.0866,0.9730
8,0.0589,0.9834,0.0792,0.9756
9,0.0512,0.9857,0.0742,0.9775
10,0.0457,0.9873,0.0829,0.9760
Final Train Accuracy: 0.9873
Final Test Accuracy: 0.9760
```

## 3.2 不同激活函数

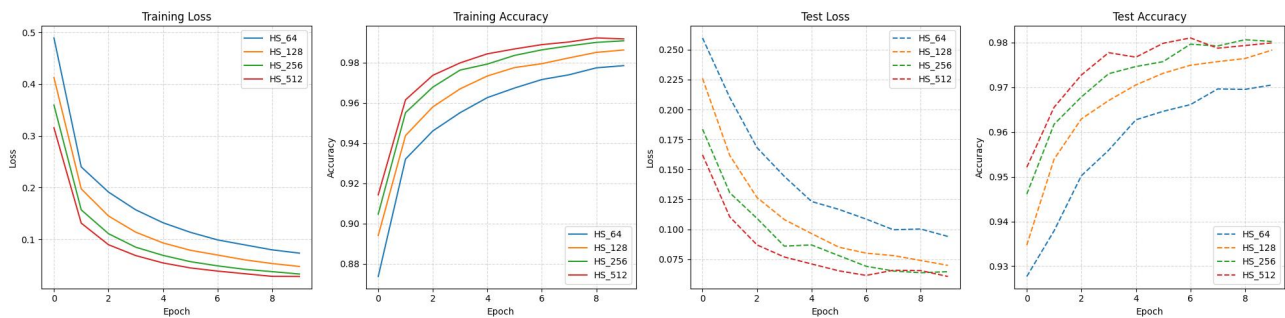
使用包含不同激活函数的网络进行训练，各网络的其他网络结构和训练条件全部相同。对于包含 ReLU（默认）、Sigmoid、Tanh、LeakyReLU 的网络，训练过程中的性能变化和分类效果如下图所示：



实验结果表明，在使用不同激活函数的网络中，ReLU、Tanh 和 LeakyReLU 激活函数在训练和测试过程中都表现出更快的收敛速度和更好的性能，训练损失和测试损失都迅速下降，训练准确率和测试准确率快速上升并稳定在较高水平；相比之下，Sigmoid 激活函数的收敛速度较慢，损失值较高，表明它在这种网络结构中可能由于梯度消失或饱和问题而性能较差。

### 3.3 不同隐藏层大小

保持默认网络结构和训练条件不变，仅改变隐藏层大小，对隐藏层大小为 64、128（默认）、256、512 的网络，训练过程中的性能变化和分类效果如下图所示：



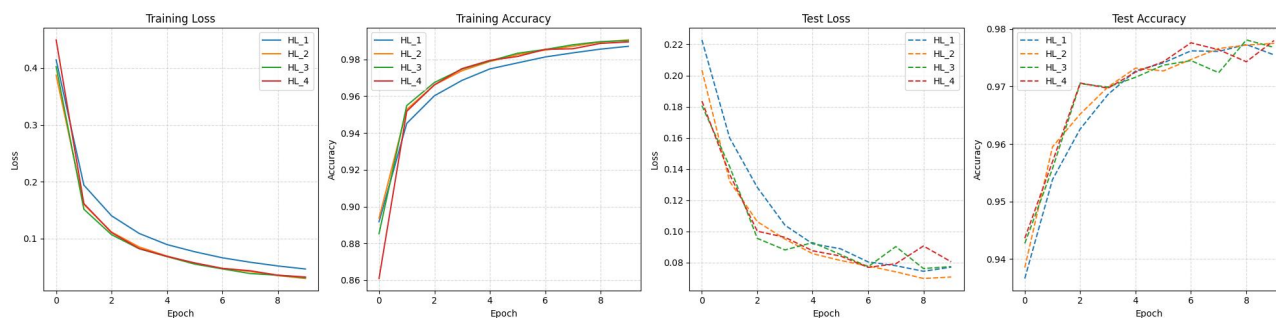
实验结果表明，随着隐藏层维数的增加，网络的收敛速度和分类效果逐渐提升。这表明隐藏层维数的增加有利于网络参数的学习，较大的隐藏层维数能有效增强网络的性能和分类效果；但在维数达到一定大小时，网络性能将收敛并不再随维度的增加显著变化。维数越大，模型参数越多，一味增大隐藏层维数，反而会徒增模型复杂程度，甚至造成网络收敛较慢、性能变差等不良后果。

### 3.4 不同隐藏层层数

使用包含不同数目的隐藏层的网络进行训练，其中，各网络在除输出层外的所有层后都使用了 ReLU 激活函数进行变换，且其他网络结构和训练条件均保持一致。对于 1 层隐藏层（默认）、2 层隐藏层、3 层隐藏层和 4 层隐藏层网络，



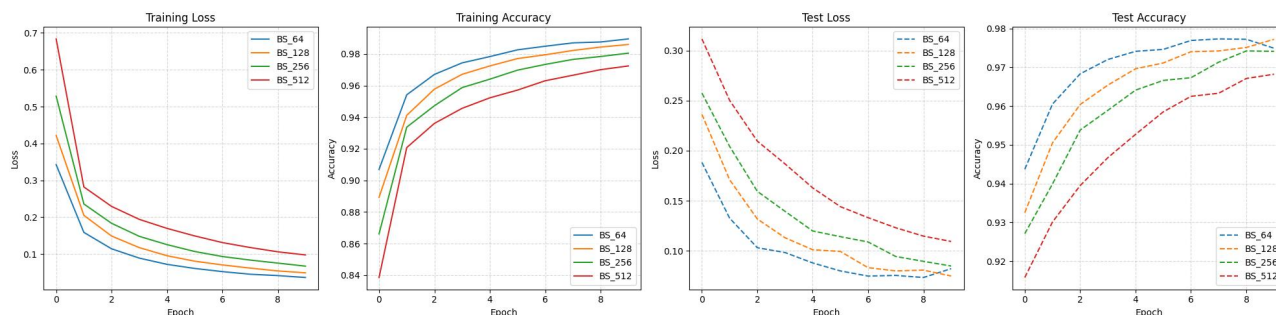
训练过程中的性能变化和分类效果如下图所示：



实验结果表明，在相同训练条件与每层 128 神经元的前提下，随着隐藏层数从 1 增加到 2/3/4 层，网络的训练损失下降更快且最终更低，训练准确率也更早进入平台期，说明更深的 MLP 具备更强的表示能力与拟合能力；但在测试集上，各模型的测试准确率整体差距很小，测试损失在中后期趋于收敛，其中 2 层隐藏层通常表现出更低、更稳定的测试损失，而 3-4 层在后期出现一定幅度的波动，呈现“训练继续变好但测试收益有限甚至略波动”的现象，反映出在 MNIST 这类相对简单任务上，单纯加深网络带来的泛化收益递减，并可能引入轻微过拟合或优化不稳定；综合而言，2-3 层隐藏层在收敛速度与泛化效果之间更均衡。

### 3.5 不同批大小

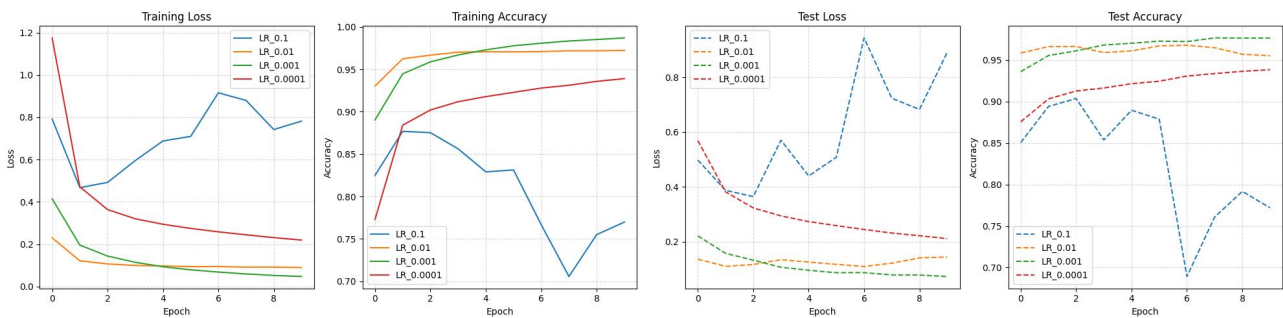
使用不同批大小的数据加载器对相同结构的网络进行训练，其他训练参数保持一致。对于采用 64、128（默认）、256 和 512 批大小的数据加载器的网络，训练过程中的性能变化和分类效果如下图所示：



实验结果表明，在本任务中，批大小越小的网络收敛速度越快。这是由于本任务的数据分布较为均匀，批大小较小时每一个 epoch 中网络参数的更新频率更大，因此能够更快地收敛。而在更为复杂的任务中，批大小较小往往会使模型陷入局部最优，或者导致模型性能变化不稳定，所以一般使用较大的批大小来避免不利现象的发生。

### 3.6 不同学习率

学习率是控制神经网络在每次参数更新时调整步长大小的超参数，它决定了模型收敛的速度和最终性能。使用不同学习率的优化器对相同结构的网络进行训练，其他训练参数保持一致。对于采用学习率为 0.1、0.01、0.001（默认）和 0.0001 优化器的网络，训练过程中的性能变化和分类效果如下图所示：



从曲线可以看出，当学习率为 0.1 时，训练损失在前期下降后出现明显反弹并大幅波动，测试损失在中后期甚至飙升，测试准确率也剧烈震荡并显著下降，说明步长过大导致更新越过最优点、训练不稳定甚至接近发散；学习率为 0.01 时，训练与测试曲线整体平滑，能够较快收敛并保持较高且稳定的测试准确率；学习率为 0.001（默认）时，损失下降最充分、测试损失最低且测试准确率最高，体现出在收敛速度与稳定性之间取得了更好的平衡；而学习率为 0.0001 虽然训练过程稳定但下降速度明显变慢，有限 epoch 内训练/测试准确率提升不足、

最终性能略低，反映出步长过小导致收敛效率偏低。该实验表明：过大学习率会引发不稳定与性能退化，过小学习率会收敛过慢。

## 4. 前向传播与反向传播过程展示

为满足解释并展示一次前向传播和一次反向传播中各层输入、输出、损失值、梯度及参数更新过程，本实验额外构造了一个 `batch_size=2` 的单步训练演示：依次执行前向传播、计算损失、反向传播与 Adam 参数更新，并将关键中间量写入 `demo.txt` 文件中。该步输入数据形状为  $(2, 784)$ ，对应标签为  $[6, 1]$ 。

```
# 1. 准备模型和数据
demo_config = {
    'name': 'default',
    'hidden_sizes': [128],
    'activation': 'relu',
    'batch_size': 2, # 为了演示方便，使用batch_size=2
    'learning_rate': 1e-3
}
model = create_model(demo_config)
# 获取一个batch的数据
demo_loader = DataLoader(train_dataset, batch_size=demo_config['batch_size'], shuffle=True)
images, labels = next(iter(demo_loader))
# 定义损失函数和优化器
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=demo_config['learning_rate'],
weight_decay=1e-4)
```

### 4.1 前向传播：逐层输入/输出与数值分布

模型结构为  $\text{Linear}(784 \rightarrow 128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128 \rightarrow 10)$ 。在实现上，通过对 `Sequential` 中每一层注册 `forward hook`，捕获每层的输入与输出张量，用于可视化与分析各层的维度变化和输出分布特征。

```
# 2. 前向传播 (Forward Pass)
# 使用hook 捕获中间层输出
layer_inputs = {}
```

```
layer_outputs = {}
def hook_fn(name):
    def hook(module, input, output):
        layer_inputs[name] = input[0].detach()
        layer_outputs[name] = output.detach()
    return hook
# 注册hook 到Sequential 中的每一层
for idx, layer in enumerate(model.network):
    layer.register_forward_hook(hook_fn(f"Layer_{idx}_{layer.__class__.__name__}"))
# 执行前向传播
outputs = model(images)
# 展示各层信息
for name in layer_inputs:
    f.write(f"[{name}]\n")
    f.write(f"  输入形状: {layer_inputs[name].shape}\n")
    f.write(f"  输出形状: {layer_outputs[name].shape}\n")
    # 简单展示部分数值统计
    f.write(f"  输出均值: {layer_outputs[name].mean():.4f},
            标准差: {layer_outputs[name].std():.4f}\n")
f.write(f"\n 最终模型输出 (Logits): \n{outputs}\n")
```

### • Layer\_0: Linear（全连接层）

输入形状 (2, 784), 输出形状 (2, 128), 输出均值约 0.0135、标准差约 0.1671。这表明线性变换将 784 维像素向量映射到 128 维隐藏表征, 输出呈近似零均值、一定方差的分布。

### • Layer\_1: ReLU（非线性激活）

输入形状 (2, 128), 输出形状 (2, 128), 输出均值约 0.0722、标准差约 0.0966。ReLU 将负值截断为 0, 使输出更偏向非负, 因此均值上升、分布更稀疏, 有利于引入非线性表达能力并改善梯度传播稳定性。

### • Layer\_2: Linear（输出层）

输入形状 (2, 128), 输出形状 (2, 10), 输出均值约 -0.0071、标准差约 0.0889。该层输出为 10 维 logits, 用于后续 Softmax + 交叉熵损失计算。

最终模型输出 logits 如 demo.txt 所示, 为一个形状 (2,10) 的张量, 每行对应一个样本对 10 类的打分。

```

----- 前向传播 -----
[Layer_0_Linear]
  输入形状: torch.Size([2, 784])
  输出形状: torch.Size([2, 128])
  输出均值: 0.0135, 标准差: 0.1671
[Layer_1_ReLU]
  输入形状: torch.Size([2, 128])
  输出形状: torch.Size([2, 128])
  输出均值: 0.0722, 标准差: 0.0966
[Layer_2_Linear]
  输入形状: torch.Size([2, 128])
  输出形状: torch.Size([2, 10])
  输出均值: -0.0071, 标准差: 0.0889

```

最终模型输出 (Logits):

```

tensor([[ 0.0456, -0.1425,  0.1644, -0.0002, -0.0200,  0.0046, -0.0553, -0.1487,
          -0.1030,  0.0520],
        [ 0.0319, -0.0841,  0.0951,  0.0691,  0.0245,  0.0689, -0.0307, -0.0386,
          -0.1613,  0.0854]], grad_fn=<AddmmBackward0>)

```

## 4.2 损失计算

```

# 3. 计算损失 (Loss Calculation)
loss = criterion(outputs, labels)
f.write(f"CrossEntropyLoss: {loss.item():.4f}\n")

```

采用 CrossEntropyLoss 计算分类损失。交叉熵在实现上等价于对 logits 先做 log\_softmax 再取真实类别的负对数似然，并对 batch 求平均。该次演示的损失值为 2.3688。

由于类别数为 10，随机猜测的期望损失约为  $\ln(10) \approx 2.3026$ 。本次损失略高于该值，说明在该随机 batch 上模型尚未对真实类别形成明显优势（符合训练早期或随机初始化时的现象）。

## 4.3 反向传播：梯度计算

```

# 4. 反向传播 (Backward Pass)
# 检查反向传播前的梯度 (应该是 None 或者 0)
f.write(f"反向传播前的梯度 (第一层权重): {model.network[0].weight.grad}\n")
loss.backward()
f.write("反向传播后的梯度范数 (Gradient Norm):\n")
for name, param in model.named_parameters():

```



```
if param.grad is not None:
    f.write(f" {name}: {param.grad.norm():.4f}\n")
else:
    f.write(f" {name}: None\n")
```

在执行 `loss.backward()` 前，第一层权重梯度为 `None`，这是因为尚未进行反向传播、梯度还未被计算与写入参数的 `grad` 字段。调用 `loss.backward()` 后，梯度从损失出发沿计算图按链式法则反向传播，依次经过输出层线性变换、ReLU 门控、隐藏层线性变换，最终得到各可训练参数的梯度。为便于展示，记录了每个参数梯度的 L2 范数：

```
----- 反向传播 -----
反向传播前的梯度（第一层权重）: None
反向传播后的梯度范数（Gradient Norm）:
network.0.weight: 2.4890
network.0.bias: 0.2661
network.2.weight: 0.8947
network.2.bias: 0.6441
```

从结果看，第一层权重梯度范数较大，说明在该 `batch` 上输入到隐藏层的映射对损失较敏感；输出层权重与偏置同样产生了可观梯度，体现了分类误差对输出打分的直接驱动。ReLU 层本身无可训练参数，但它会通过正区间梯度为 1、负区间梯度为 0 的门控机制影响梯度是否能传回前一层，从而影响 `network.0` 的梯度大小。

## 4.4 参数更新

```
# 5. 参数更新 (Parameter Update)
# 选取一个参数观察更新情况（例如第一层权重的第一个元素）
param_to_watch = model.network[0].weight[0, 0].item()
f.write(f"更新前参数值 (Layer 0, weight[0,0]): {param_to_watch:.6f}\n")
optimizer.step()
param_updated = model.network[0].weight[0, 0].item()
f.write(f"更新后参数值 (Layer 0, weight[0,0]): {param_updated:.6f}\n")
f.write(f"参数变化量: {param_updated - param_to_watch:.6f}\n")
```

优化器使用 `Adam(lr=1e-3, weight_decay=1e-4)`。在调用 `optimizer.step()` 时，

Adam 会利用本步梯度以及历史一阶/二阶矩估计来生成自适应步长，并结合权重衰减对参数进行更新。为直观验证参数确实被更新，演示中选取第一层权重矩阵的一个元素 `weight[0,0]` 进行对比：

```
----- 参数更新 -----  
更新前参数值 (Layer 0, weight[0,0]): 0.016173  
更新后参数值 (Layer 0, weight[0,0]): 0.015179  
参数变化量: -0.000994
```

该结果表明：反向传播得到的梯度经过 Adam 规则处理后，参数沿着使损失下降的方向发生了可测的数值变化，从而完成一次完整的训练闭环。

正式训练循环中通常还会在每次反向传播前调用 `optimizer.zero_grad()` 清空上一轮梯度，避免梯度累积；本演示中新建模型且演示单步更新，因此在反向传播前梯度为 `None` 的现象也能清晰体现梯度生成发生在 `backward` 之后。

## 5. 总结与体会

本实验围绕“多层感知机的前向与反向传播机制探究”展开，通过搭建并训练 MLP，我对从前向计算到损失构建、再到反向传播生成梯度并由优化器更新参数的完整链路形成了更清晰的理解。实验过程中也体会到模型结构与关键超参数会显著影响收敛稳定性与泛化表现，因此在实际建模中需要在表达能力、训练效率与过拟合风险之间做权衡。